VELLORE INSTITUTE OF TECHNOLOGY, CHENNAI

FALL SEMESTER 2026–2027

BACSE202 – DATABASE MANAGEMENT SYSTEMS

Digital Assignment – Technical Report

RideGraph

A Graph-Database Ride-Hailing Platform using Neo4j, Flask and React

Name	Register Number
Akhil Gokul	25BCE5009
Praveen Sakthi S	25BCE1605
Prakash V	25BCE1769

Faculty: Dr. S. Geetha

Slot: D1 + TD1 Class: CH202670101707

GitHub: https://github.com/akhilax/ridegraph

Demo video: https://drive.google.com/file/d/11rI7FD1uJiBr3lQKsv9nnGIfxgUtdqhg/view

Abstract

Ride-hailing services depend on relationships between data: which rider requested which ride, which driver in which zone accepted it, where it started and ended, and how it was paid and rated. In a relational database each of these questions becomes a chain of joins. **RideGraph** is a working ride-hailing web application for Chennai that stores this domain as a **property graph in Neo4j**, so the core operations become short graph traversals.

The system supports three roles. A **rider** registers, gets a fare estimate, requests a ride, pays by cash, card or UPI, and rates the driver. A **driver** sees open requests only in their own zone, accepts and completes rides, and tracks earnings. An **admin** views revenue, rides per zone, top drivers and cancellations. The data model was designed as an ER diagram with 13 entities and relationships. It was converted to a relational schema, normalized to 3NF, and then mapped to 11 node labels and 12 relationship types. The application uses a Python Flask REST API with parameterised Cypher queries, uniqueness constraints on every key, salted password hashing and conditional updates that prevent double booking. The interface is a React single-page app. An automated test script exercises the full ride life cycle, along with error cases such as duplicate registration, invalid input, wrong zone and repeated actions.

Keywords: Graph database, Neo4j, Cypher, ride-hailing, ER modelling, normalization, property graph, Flask REST API, React, data integrity

Table of Contents

Abstract....2
Table of Contents....3
1. Introduction....5
1.1 Problem Statement....5
1.2 Intended Users....5
1.3 Objectives....5
1.4 Scope....6
2. Existing Systems....7
2.1 Uber....7
2.2 Ola....7
2.3 Rapido....8
2.4 Comparison and Gap....9
3. Requirements and Proposed System Architecture....10
3.1 Functional Requirements....10
3.2 Non-Functional Requirements....10
3.3 Hardware and Software Requirements....10
3.4 Proposed System Architecture....11
4. ER Diagram and Relational Schema....12
4.1 ER Diagram....12
4.2 Entities and Relationships....12
4.3 Relational Schema....13
4.4 Mapping the Schema to the Neo4j Graph....14
5. Keys, Constraints, Functional Dependencies and Normalization....15
5.1 Keys....15
5.2 Integrity Constraints....15
5.3 Initial Relation....15
5.4 Functional Dependencies....16
5.5 First Normal Form (1NF)....16
5.6 Second Normal Form (2NF)....17
5.7 Third Normal Form (3NF)....17
5.8 Justified Denormalization....18
6. Implementation Details....19
6.1 Database Management System....19
6.2 Application Technologies....19
6.3 Database Connectivity....20
6.4 Project Structure....20
6.5 Sample Data....20
7. Important Queries and Implemented Features....21
7.1 Feature 1 – Registration and Login....21
7.2 Feature 2 – Fare Estimate and Ride Request (INSERT)....21
7.3 Feature 3 – Nearby Drivers and Open Requests (traversal / JOIN)....21
7.4 Feature 4 – Accept, Complete, Pay (UPDATE, concurrency-safe)....22
7.5 Feature 5 – Rating with Automatic Average....22
7.6 Feature 6 – Cancellation....22
7.7 Feature 7 – Admin Statistics (aggregation / GROUP BY)....23
7.8 Subqueries, Views, Transactions and DELETE....23
8. Testing, Results and Discussion....24
8.1 Testing Approach....24

8.2 Test Cases....24
8.3 Results on the Sample Data....25
8.4 Screenshots....26
8.5 Discussion....32
9. Conclusion, Limitations and Future Enhancements....33
9.1 Conclusion....33
9.2 Limitations....33
9.3 Future Enhancements....33
References....34

(If the page numbers are not shown, right-click the table in Word and choose Update Field.)

1. Introduction

Ride-hailing apps such as Uber, Ola and Rapido match a rider who needs a trip with a nearby driver who is free, price the trip, track it through its life cycle and settle payment and feedback afterwards. Every step reads or changes **connections between records**: rider → ride → driver → vehicle, ride → pickup/dropoff location → zone, ride → payment/rating/promo/cancellation.

RideGraph implements this workflow as a complete database-backed web application. The data is held in Neo4j, a native graph database, where these connections are stored as first-class relationships. The project covers the full database life cycle: requirements analysis, ER modelling, relational schema design with keys and constraints, normalization, implementation, application connectivity, testing and documentation.

1.1 Problem Statement

Design and build a ride-hailing system in which riders can book rides and drivers can find and serve requests in their area, and in which the platform keeps a consistent record of every ride, payment, rating and cancellation. The system must answer location- and relationship-heavy questions efficiently. Examples are "which available drivers are in this pickup zone?", "which open requests can this driver accept?" and "how much has this driver earned?". It must also enforce integrity rules such as one driver per ride, one payment and one rating per ride, and no double booking.

1.2 Intended Users

- **Riders**: book rides, apply promo codes, pay, rate and view their history.
- **Drivers**: see and accept requests in their zone, complete rides, and view their earnings and rating.
- **Administrator**: monitors platform statistics (rides, revenue, zones, drivers, cancellations).

1.3 Objectives

1. Model the ride-hailing domain with an ER diagram and derive a relational schema with primary, foreign and candidate keys.
2. Normalize the relations to Third Normal Form and justify any deliberate denormalization.
3. Implement the database in Neo4j with uniqueness constraints and indexes, and load realistic synthetic data.
4. Build a three-tier web application (React → Flask → Neo4j) with at least five complete end-to-end features.
5. Validate all input and handle invalid operations, duplicates, missing records and constraint violations with clear error messages.
6. Test the system with documented test cases and automated end-to-end tests.

1.4 Scope

In scope: registration and login for riders and drivers; fare estimation with ride types and promo codes; ride request, zone-based matching, acceptance, completion and cancellation; payment recording; ratings with automatic driver-rating recalculation; ride history and driver earnings; an admin statistics dashboard; six Chennai zones with fifteen named locations.

Out of scope: live GPS tracking and maps, road-network routing (distances are straight-line), real payment gateways, token-based authentication, surge pricing and push notifications. These are listed as future enhancements.

2. Existing Systems

Three widely used ride-hailing platforms in India were studied to identify the features RideGraph should include.

2.1 Uber

URL: https://www.uber.com/in/en/

A global ride-hailing platform offering cars and autos across multiple categories (e.g., Uber Go, Premier, Auto, Moto). Riders get an upfront fare, driver details and live tracking, and pay in-app or in cash. Riders and drivers rate each other after every trip.

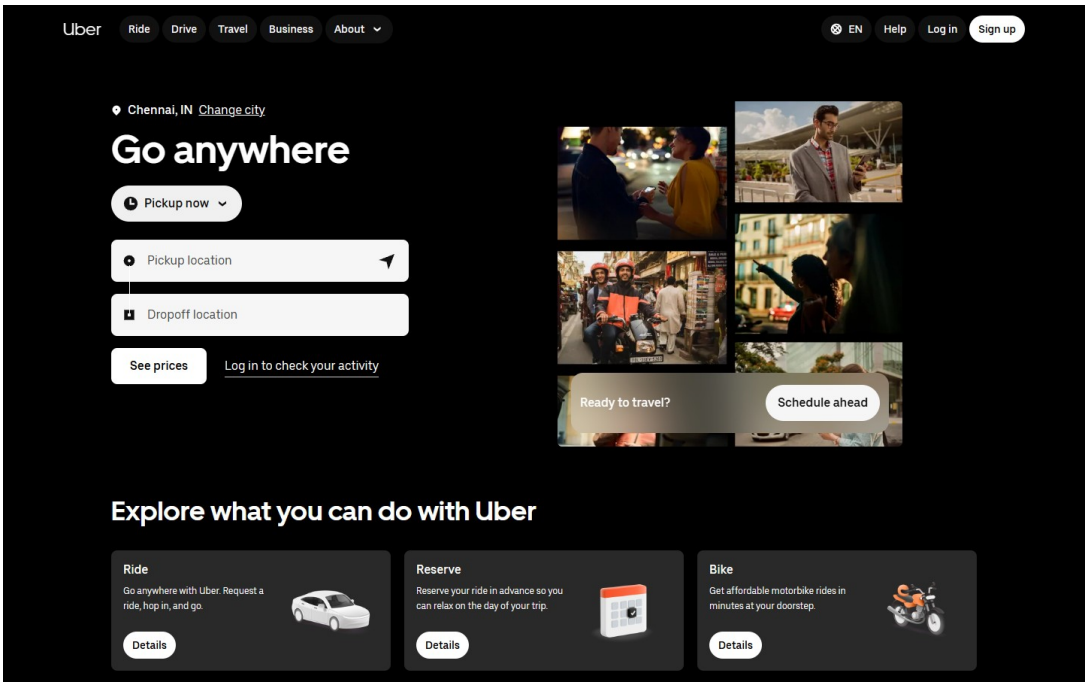


Figure 2.1: Uber – home page (https://www.uber.com/in/en/, captured 01-11-2026)

2.2 Ola

URL: https://www.olacabs.com/

An Indian ride-hailing platform with Mini, Prime Sedan, Auto and Bike categories, coupon codes, multiple payment options (cash, UPI, Ola Money) and trip history.

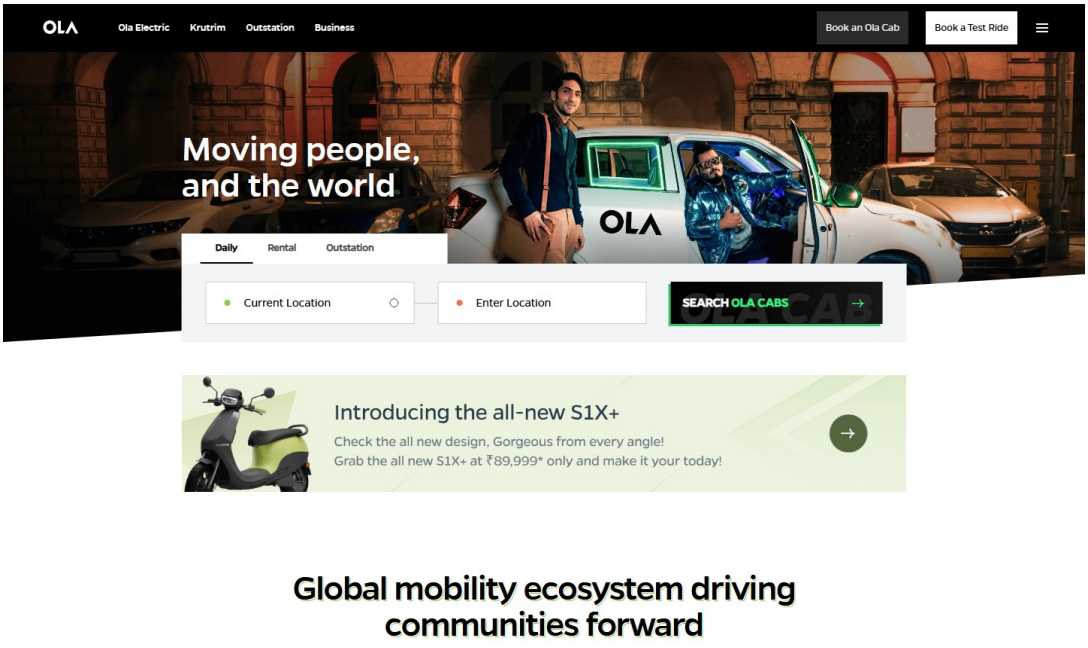


Figure 2.2: Ola – home page (https://www.olacabs.com/, captured 01-11-2026)

2.3 Rapido

URL: https://www.rapido.bike/

A platform focused on bike taxis and autos, with low per-km fares, zone-based captain availability and a simple booking flow.

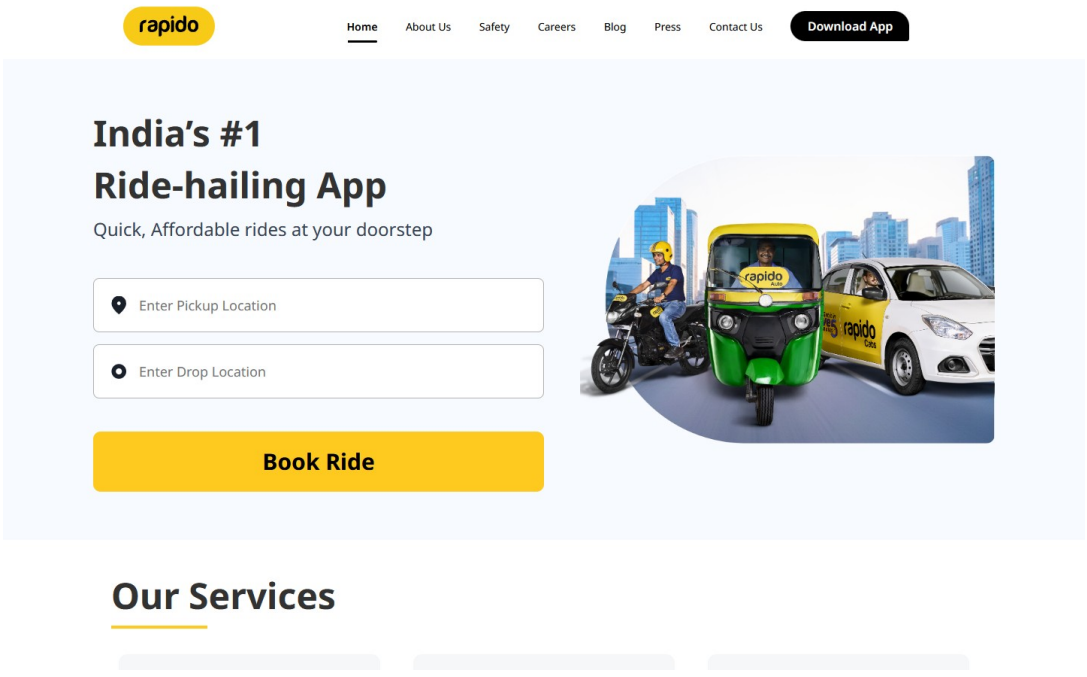


Figure 2.3: Rapido – home page (https://www.rapido.bike/, captured 01-11-2026)

2.4 Comparison and Gap

Feature	Uber	Ola	Rapido	RideGraph
Multiple ride types	Yes	Yes	Yes	Yes (Economy, Premium, Auto, Bike)
Fare estimate before booking	Yes	Yes	Yes	Yes (base + per-km, haversine)
Promo codes	Yes	Yes	Yes	Yes
Nearby-driver matching	Yes (GPS)	Yes (GPS)	Yes (GPS)	Yes (zone graph traversal)
Ratings	Two-way	Two-way	Two-way	Rider → driver, avg. recalculated
Admin analytics	Internal	Internal	Internal	Yes (dashboard)
Data model open for study	No	No	No	Yes – documented graph model

Table 2.1: Feature comparison with existing systems

Commercial platforms are closed systems whose data design cannot be studied. RideGraph reproduces their core workflow on a small scale with a fully documented database design. This shows how a graph database models the relationship-heavy parts of the problem (matching, history, statistics) more directly than a chain of relational joins.

3. Requirements and Proposed System Architecture

3.1 Functional Requirements

ID	Requirement
FR1	Riders register with name, email, phone and password (≥ 6 characters); email must be unique.
FR2	Drivers register with name, phone, licence number, password, zone and vehicle (plate, model, colour); phone and licence must be unique.
FR3	Riders, drivers and the admin log in; a wrong login returns one generic message.
FR4	A rider gets a fare estimate for pickup, dropoff, ride type and optional promo code.
FR5	A rider requests a ride; the system stores it with status requested.
FR6	The system lists available drivers in the pickup zone, best rated first.
FR7	A driver sees open requests whose pickup lies in their zone, and can accept one if available.
FR8	The accepting driver completes the ride; a pending payment is created.
FR9	The rider pays (cash / card / UPI) once and rates the ride (1–5) once.
FR10	Rider or driver can cancel a requested/accepted ride with a reason.
FR11	Riders view ride history; drivers view their rides and total earnings.
FR12	The admin views totals, revenue, rides per zone, top drivers and cancellations.

Table 3.1: Functional requirements

3.2 Non-Functional Requirements

- **Integrity:** unique keys on every entity; one payment, one rating and at most one cancellation per ride; no double acceptance.
- **Security:** passwords stored only as salted hashes; all queries parameterised (no Cypher injection); database password kept in an untracked `.env` file.
- **Usability:** clear validation messages and correct HTTP status codes (400, 401, 403, 404, 409, 503).
- **Portability:** one-click run scripts for Windows and macOS; the pre-built frontend means Node.js is not required to run the app.
- **Re-runnable setup:** seed script uses MERGE so it can be run repeatedly without duplicating data.

3.3 Hardware and Software Requirements

Item	Requirement
Processor / RAM	Any modern x64 CPU, 8 GB RAM recommended (Neo4j + browser)
Operating system	Windows 10/11 or macOS
Database	Neo4j 5.x via Neo4j Desktop 2.x (Community features)
Backend	Python 3.10+, Flask, neo4j driver, python-dotenv, flask-cors, Werkzeug
Frontend	React 19, Vite, React Router, Tailwind CSS, shadcn/ui, Motion (Node.js only to rebuild)
Client	Any modern web browser

Table 3.2: Hardware and software requirements

3.4 Proposed System Architecture

RideGraph follows a **three-tier architecture**. The browser never connects to the database directly. All reads and writes pass through the Flask API, which validates input, applies the business rules and runs parameterised Cypher over the Bolt protocol.

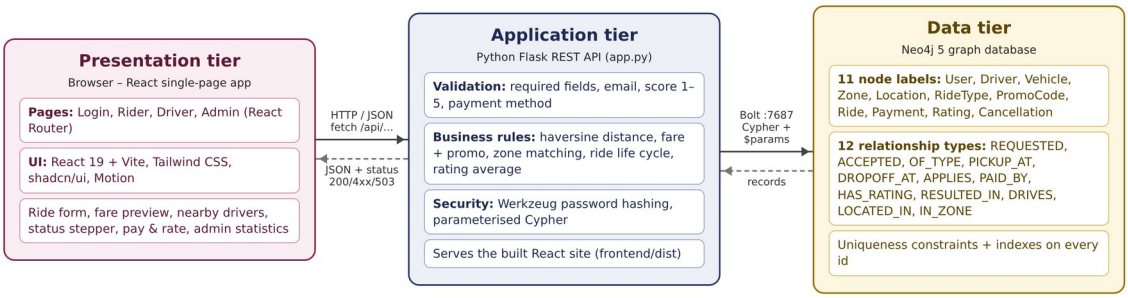


Figure 3.1: Three-tier architecture of RideGraph

- **Presentation tier:** a React single-page application with Login, Rider, Driver and Admin pages. Vite builds it into static files, which Flask serves.
- **Application tier:** a Flask REST API (`app.py`). It computes distance and fare, enforces who may accept, complete, cancel, pay or rate, and returns JSON with proper status codes.
- **Data tier:** a Neo4j graph database accessed through the official Python driver (`db.py`). All Cypher statements are kept in `queries.py`.

4. ER Diagram and Relational Schema

4.1 ER Diagram

The ER diagram has thirteen entity types centred on **RIDES**. A user requests many rides. A driver accepts many rides, drives one vehicle and is located in one zone. Each ride has one ride type, one pickup and one dropoff location, and optionally one promo code. A ride has at most one payment, one rating and one cancellation.

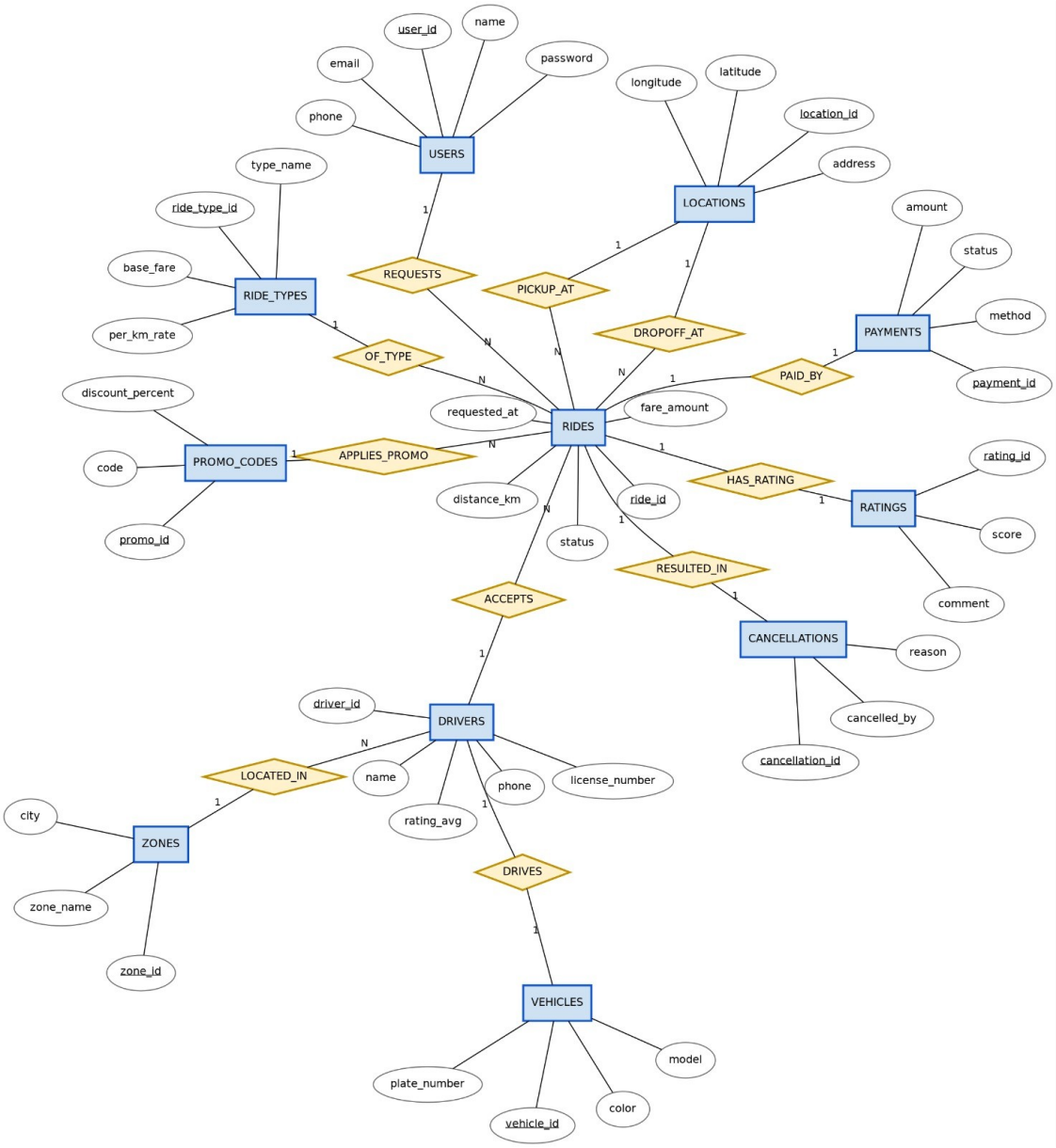


Figure 4.1: ER diagram of RideGraph

4.2 Entities and Relationships

Relationship	Entities	Cardinality	Participation
REQUESTS	USERS – RIDES	1 : N	Every ride must have a user (total on RIDES)

Relationship	Entities	Cardinality	Participation
ACCEPTS	DRIVERS – RIDES	1 : N	Partial – a requested ride has no driver yet
OF_TYPE	RIDE_TYPES – RIDES	1 : N	Total on RIDES
PICKUP_AT	LOCATIONS – RIDES	1 : N	Total on RIDES
DROPOFF_AT	LOCATIONS – RIDES	1 : N	Total on RIDES
APPLIES_PROMO	PROMO_CODES – RIDES	1 : N	Partial (promo is optional)
PAID_BY	RIDES – PAYMENTS	1 : 1	Total on PAYMENTS
HAS_RATING	RIDES – RATINGS	1 : 1	Total on RATINGS
RESULTED_IN	RIDES – CANCELLATIONS	1 : 1	Total on CANCELLATIONS
DRIVES	DRIVERS – VEHICLES	1 : 1	Total on both
LOCATED_IN	ZONES – DRIVERS	1 : N	Total on DRIVERS
IN_ZONE*	ZONES – LOCATIONS	1 : N	Total on LOCATIONS

*Table 4.1: Relationship sets (*IN_ZONE links each location to its zone and is used for zone matching)*

4.3 Relational Schema

The ER diagram converts to the following relations. Primary keys are **bold**, and FK marks a foreign key. In the 1:1 relationships PAID_BY, HAS_RATING and RESULTED_IN, the foreign key goes on the dependent side (Payment, Rating, Cancellation) with a UNIQUE constraint. For DRIVES it goes on Vehicles.

Relation	Attributes
ZONES	zone_id, zone_name, city
LOCATIONS	location_id, address, latitude, longitude, zone_id (FK → ZONES)
USERS	user_id, name, email (UNIQUE), phone, password_hash
DRIVERS	driver_id, name, phone (UNIQUE), license_number (UNIQUE), rating_avg, status, zone_id (FK → ZONES)
VEHICLES	vehicle_id, plate_number (UNIQUE), model, color, driver_id (FK → DRIVERS, UNIQUE)
RIDE_TYPES	ride_type_id, type_name (UNIQUE), base_fare, per_km_rate
PROMO_CODES	promo_id, code (UNIQUE), discount_percent
RIDES	ride_id, requested_at, status, distance_km, fare_amount, user_id (FK → USERS), driver_id (FK → DRIVERS, NULL until accepted), ride_type_id (FK), pickup_location_id (FK → LOCATIONS), dropoff_location_id (FK → LOCATIONS), promo_id (FK, NULL)
PAYMENTS	payment_id, ride_id (FK → RIDES, UNIQUE), amount, method, status
RATINGS	rating_id, ride_id (FK → RIDES, UNIQUE), score, comment
CANCELLATIONS	cancellation_id, ride_id (FK → RIDES, UNIQUE), reason, cancelled_by

Table 4.2: Relational schema

4.4 Mapping the Schema to the Neo4j Graph

The relational schema is implemented as a property graph using the standard mapping. Each relation becomes a **node label**, and each tuple becomes a node whose attributes are its properties. Each **foreign key becomes a relationship**, so the join it would require is stored directly. Foreign-key columns therefore disappear from the nodes. For example, `RIDES.user_id` becomes `(:User)-[:REQUESTED]->(:Ride)`, and `RIDES.pickup_location_id` becomes `(:Ride)-[:PICKUP_AT]->(:Location)`.

Relational foreign key	Graph relationship
RIDES.user_id → USERS	(:User)-[:REQUESTED]->(:Ride)
RIDES.driver_id → DRIVERS	(:Driver)-[:ACCEPTED]->(:Ride)
RIDES.ride_type_id → RIDE_TYPES	(:Ride)-[:OF_TYPE]->(:RideType)
RIDES.pickup_location_id / dropoff_location_id	(:Ride)-[:PICKUP_AT]->(:Location), (:Ride)-[:DROPOFF_AT]->(:Location)
RIDES.promo_id → PROMO_CODES	(:Ride)-[:APPLIES]->(:PromoCode)
PAYMENTS.ride_id / RATINGS.ride_id / CANCELLATIONS.ride_id	(:Ride)-[:PAID_BY]->(:Payment), -[:HAS_RATING]->(:Rating), -[:RESULTED_IN]->(:Cancellation)
VEHICLES.driver_id → DRIVERS	(:Driver)-[:DRIVES]->(:Vehicle)
DRIVERS.zone_id, LOCATIONS.zone_id → ZONES	(:Driver)-[:LOCATED_IN]->(:Zone), (:Location)-[:IN_ZONE]->(:Zone)

Table 4.3: Foreign keys mapped to graph relationships

5. Keys, Constraints, Functional Dependencies and Normalization

5.1 Keys

Relation	Primary key	Candidate / unique keys	Foreign keys
USERS	user_id	email	–
DRIVERS	driver_id	phone, license_number	zone_id
VEHICLES	vehicle_id	plate_number, driver_id	driver_id
ZONES	zone_id	–	–
LOCATIONS	location_id	–	zone_id
RIDE_TYPES	ride_type_id	type_name	–
PROMO_CODES	promo_id	code	–
RIDES	ride_id	–	user_id, driver_id, ride_type_id, pickup_location_id, dropoff_location_id, promo_id
PAYMENTS	payment_id	ride_id	ride_id
RATINGS	rating_id	ride_id	ride_id
CANCELLATIONS	cancellation_id	ride_id	ride_id

Table 5.1: Primary, candidate and foreign keys

5.2 Integrity Constraints

- **Entity integrity:** every `*_id` is unique and non-null. In Neo4j this is a uniqueness constraint per label, which also creates an index.
- **Referential integrity:** a ride can only be created by `MATCH`ing an existing user, ride type and both locations. If any of them is missing, nothing is created and the API returns 404. A driver can only register in an existing zone.
- **Domain constraints:** Ride.status ∈ {requested, accepted, completed, cancelled}; Driver.status ∈ {available, busy}; Payment.method ∈ {cash, card, upi}; Payment.status ∈ {pending, paid}; Rating.score is an integer from 1 to 5; Cancellation.cancelled_by ∈ {rider, driver}; pickup ≠ dropoff; password length ≥ 6.
- **Cardinality / business constraints:** at most one rating per ride (`WHERE NOT (r)-[:HAS_RATING]->()`); a ride can only be paid once; a ride can only be accepted while it is still `requested` and the driver is `available` in the same zone.

Neo4j Community Edition supports uniqueness constraints but not property-existence or property-type constraints. The domain rules are therefore enforced in the Flask API before any write, and they are also written into the Cypher patterns so that an invalid update matches nothing.

5.3 Initial Relation

Normalization starts from the single unnormalized record that a ride receipt would produce, with the rider's rides as a repeating group:

```
RIDE_RECORD( user_id, user_name, email, phone,
```

```
  { ride_id, requested_at, status, distance_km, fare_amount,
    driver_id, driver_name, driver_phone, license_number, rating_avg,
driver_status,
    vehicle_id, plate_number, model, color, driver_zone_id, driver_zone_name,
city,
    ride_type_id, type_name, base_fare, per_km_rate,
    pickup_location_id, pickup_address, pickup_lat, pickup_lon, pickup_zone_id,
pickup_zone_name,
    dropoff_location_id, dropoff_address, dropoff_lat, dropoff_lon,
dropoff_zone_id,
    promo_id, code, discount_percent,
    payment_id, amount, method, payment_status,
    rating_id, score, comment,
    cancellation_id, reason, cancelled_by } )
```

5.4 Functional Dependencies

#	Functional dependency
FD1	user_id → user_name, email, phone, password_hash
FD2	email → user_id (candidate key)
FD3	ride_id → requested_at, status, distance_km, fare_amount, user_id, driver_id, ride_type_id, pickup_location_id, dropoff_location_id, promo_id
FD4	driver_id → driver_name, driver_phone, license_number, rating_avg, driver_status, zone_id, vehicle_id
FD5	license_number → driver_id; driver_phone → driver_id (candidate keys)
FD6	vehicle_id → plate_number, model, color; plate_number → vehicle_id
FD7	zone_id → zone_name, city
FD8	location_id → address, latitude, longitude, zone_id
FD9	ride_type_id → type_name, base_fare, per_km_rate
FD10	promo_id → code, discount_percent; code → promo_id
FD11	payment_id → ride_id, amount, method, payment_status; ride_id → payment_id
FD12	rating_id → ride_id, score, comment; ride_id → rating_id
FD13	cancellation_id → ride_id, reason, cancelled_by; ride_id → cancellation_id

Table 5.2: Functional dependencies

5.5 First Normal Form (1NF)

RIDE_RECORD is not in 1NF because the ride details form a repeating group: one user has many rides. The group is removed by repeating the user attributes on every ride row, so every attribute holds a single atomic value. The key of the flattened relation is the composite **(user_id, ride_id)**.

```
R1( user_id, ride_id, user_name, email, phone, requested_at, status, ...all ride,
    driver, vehicle, zone, type, location, promo, payment, rating, cancellation
attributes )
```

R1 is in 1NF but has redundancy. A user's name and email are repeated on every ride, and a ride type's fare rates are repeated on every ride of that type. This causes update, insertion and deletion anomalies. For example, a new ride type cannot be stored until somebody books it.

5.6 Second Normal Form (2NF)

2NF forbids non-key attributes that depend on only part of the composite key. In R1, `user_name, email, phone` depend on `user_id` alone (FD1). Every ride attribute depends on `ride_id` alone (FD3), and `ride_id` also determines `user_id`. These partial dependencies are removed by decomposition:

```
USERS( user_id, user_name, email, phone, password_hash )
R2( ride_id, user_id, requested_at, status, distance_km, fare_amount, driver_id,
driver_name, ...,
    ride_type_id, type_name, base_fare, ..., pickup_location_id,
pickup_address, ...,
    promo_id, code, ..., payment_id, amount, ..., rating_id, score, ...,
cancellation_id, reason, ... )
```

USERS has a single-attribute key, so it is in 2NF. R2 now has the single-attribute key `ride_id`, so no partial dependency is possible and R2 is in 2NF.

5.7 Third Normal Form (3NF)

3NF forbids **transitive dependencies**, where a non-key attribute depends on another non-key attribute. R2 has several, all through foreign-key attributes:

- ride_id → driver_id → driver_name, license_number, rating_avg, zone_id, vehicle_id (FD4), and further vehicle_id → plate_number, model, color (FD6) and zone_id → zone_name, city (FD7)
- ride_id → ride_type_id → type_name, base_fare, per_km_rate (FD9)
- ride_id → pickup_location_id / dropoff_location_id → address, latitude, longitude, zone_id (FD8) → zone_name (FD7)
- ride_id → promo_id → code, discount_percent (FD10)
- ride_id → payment_id / rating_id / cancellation_id → their attributes (FD11–FD13)

Each determinant is moved into its own relation, and only its key is kept as a foreign key. This gives the final 3NF relations:

Relation	Key	Dependencies it holds
USERS(user_id, name, email, phone, password_hash)	user_id (alt: email)	FD1, FD2
ZONES(zone_id, zone_name, city)	zone_id	FD7
LOCATIONS(location_id, address, latitude, longitude, zone_id)	location_id	FD8
DRIVERS(driver_id, name, phone, license_number, rating_avg, status, zone_id)	driver_id (alt: phone, license_number)	FD4, FD5
VEHICLES(vehicle_id, plate_number, model, color, driver_id)	vehicle_id (alt: plate_number, driver_id)	FD6
RIDE_TYPES(ride_type_id, type_name, base_fare, per_km_rate)	ride_type_id	FD9
PROMO_CODES(promo_id, code, discount_percent)	promo_id (alt: code)	FD10
RIDES(ride_id, requested_at, status, distance_km, fare_amount, user_id, driver_id, ride_type_id, pickup_location_id, dropoff_location_id, promo_id)	ride_id	FD3

Relation	Key	Dependencies it holds
PAYMENTS(payment_id, ride_id, amount, method, status)	payment_id (alt: ride_id)	FD11
RATINGS(rating_id, ride_id, score, comment)	rating_id (alt: ride_id)	FD12
CANCELLATIONS(cancellation_id, ride_id, reason, cancelled_by)	cancellation_id (alt: ride_id)	FD13

Table 5.3: Final relations in 3NF

In every final relation, each non-trivial dependency has a candidate key on its left-hand side, so the relations are in 3NF and also in **BCNF**. The decomposition is lossless, because every split shares a key with the relation it came from. It is also dependency-preserving, because every FD is contained in one relation. These eleven relations are exactly the eleven node labels in the Neo4j implementation.

5.8 Justified Denormalization

Four stored attributes can be derived from other data. They are kept deliberately:

- **RIDES.distance_km** can be derived from the two locations' coordinates. It is stored so that history and statistics do not recompute the haversine formula on every read.
- **RIDES.fare_amount** is a price snapshot taken at booking time. If base_fare, per_km_rate or a promo changes later, past rides must keep the price that was actually charged. Recomputing it would rewrite history.
- **PAYMENTS.amount** copies the fare when the payment is created, as a separate financial record that is independent of later edits to the ride.
- **DRIVERS.rating_avg** is the average of the driver's ratings. It is stored so that "nearby drivers, best rated first" can sort without aggregating every rating. It is recalculated by `UPDATE_DRIVER_RATING` each time a rating is added, so it cannot drift.

6. Implementation Details

6.1 Database Management System

The database is **Neo4j 5**, run locally through Neo4j Desktop 2.x as the instance `ridegraph-db`. Neo4j was chosen, with the faculty's approval, because the core operations of a ride-hailing system are traversals over relationships. Neo4j stores relationships as direct pointers between nodes (index-free adjacency). So finding the drivers in a pickup zone follows a few links instead of computing joins over whole tables, and the cost depends on the size of the neighbourhood rather than the size of the database.

Constraints and indexes are created by `src/backend/seed.py` and by `database/schema.cypher`. There is one uniqueness constraint per label on its id, plus unique constraints on email, phone, licence, plate and promo code, and indexes on the frequently filtered properties `Ride.status`, `Ride.requested_at` and `Driver.status`.

```
CREATE CONSTRAINT ride_ride_id IF NOT EXISTS FOR (n:Ride) REQUIRE n.ride_id IS
UNIQUE;
CREATE CONSTRAINT user_email_unique IF NOT EXISTS FOR (u:User) REQUIRE u.email IS
UNIQUE;
CREATE INDEX ride_status_idx IF NOT EXISTS FOR (r:Ride) ON (r.status);
```

Relational (SQL) version. The same 3NF design is also provided as executable SQL scripts, tested with SQLite 3: `database/schema.sql` creates the 11 tables with PRIMARY KEY, FOREIGN KEY (with ON DELETE CASCADE for the 1:1 dependents), UNIQUE, NOT NULL and CHECK constraints, six indexes and the view `ride_summary`; `database/sample_data.sql` loads the same synthetic data inside a transaction; and `database/queries.sql` runs a full ride life cycle with INSERT, UPDATE (in BEGIN… COMMIT transactions), SELECT and DELETE, followed by joins, GROUP BY/HAVING, subqueries (scalar, EXISTS, NOT EXISTS) and the view. Its last five statements are deliberate violations (duplicate primary key, duplicate email, score 9, unknown user, same pickup and drop-off), and each is rejected by the corresponding constraint. The SQL version returns the same results as Neo4j (24 rides, 21 completed, ₹1,973.40 revenue).

6.2 Application Technologies

Component	Technology	Role
Backend API	Python 3, Flask, flask-cors	REST endpoints, validation, business rules, serving the frontend
Database driver	neo4j (official Python driver)	Bolt connection pool, transactions
Configuration	python-dotenv	Reads NEO4J_URI / USER / PASSWORD from .env
Security	Werkzeug	Salted password hashing (scrypt)
Frontend	React 19, React Router, Vite	Single-page app: Login, Rider, Driver, Admin pages
UI	Tailwind CSS, shadcn/ui, Motion	Components, theming (light/dark), animations

Table 6.1: Application technology stack

6.3 Database Connectivity

`db.py` creates one driver object for the whole application, which manages a pool of Bolt connections. It exposes two helper functions, `run_read` and `run_write`. Each call runs in its own transaction, routed as a read or a write.

```
load_dotenv(os.path.join(os.path.dirname(os.path.abspath(__file__)), ".env"))
URI = os.getenv("NEO4J_URI", "bolt://localhost:7687")
USER = os.getenv("NEO4J_USER", "neo4j")
PASSWORD = os.getenv("NEO4J_PASSWORD", "")
DATABASE = os.getenv("NEO4J_DATABASE", "neo4j")
driver = GraphDatabase.driver(URI, auth=(USER, PASSWORD))

def run_read(query, **params):
    records, _, _ = driver.execute_query(
        query, parameters_=params, database_=DATABASE, routing_="r")
    return [r.data() for r in records]

def run_write(query, **params):
    records, _, _ = driver.execute_query(
        query, parameters_=params, database_=DATABASE, routing_="w")
    return [r.data() for r in records]
```

Every query passes user input as `$parameters` and never joins it into the query text, which prevents Cypher injection. If Neo4j is stopped or the password is wrong, the driver raises `ServiceUnavailable` or `AuthError`. Flask turns this into a 503 response with a clear message.

6.4 Project Structure

```
RideGraph/
├── README.md
├── database/   schema / sample_data / queries  (.cypher for Neo4j, .sql for SQL)
├── src/backend/   app.py (API), db.py (connection), queries.py (all Cypher),
│                  seed.py (constraints + demo data), utils.py (haversine, fare)
├── src/frontend/  React source (src/) and built site (dist/)
├── tests/     test_api.py, TEST_CASES.md
├── screenshots/, report/, presentation/, docs/
└── run_windows.bat, run_mac.command, requirements.txt
```

6.5 Sample Data

All data is synthetic and covers Chennai: 6 zones, 15 named locations, 4 ride types (Economy ₹40 + ₹12/km, Premium ₹80 + ₹20/km, Auto ₹30 + ₹10/km, Bike ₹20 + ₹6/km), 3 promo codes (WELCOME20, RIDE10, SAVE15), 10 riders, 11 drivers with vehicles, and 24 historical rides: 21 completed with payments and ratings, and 3 cancelled. The demo accounts use fixed demo passwords that are stored only as hashes.

7. Important Queries and Implemented Features

The application has more than five complete end-to-end features. Each one goes from a React screen, through a Flask endpoint, to one or more Cypher queries and back. The key queries are shown below. All of the application's queries are in `src/backend/queries.py`, and runnable versions are in `database/queries.cypher`.

7.1 Feature 1 – Registration and Login

`POST /api/register` checks the required fields, the email format and the password length, rejects a duplicate email, phone or licence with 409, hashes the password and creates the node. A driver is created together with their vehicle and zone link in a single query:

```
MATCH (z:Zone {zone_id: $zone_id})
CREATE (d:Driver {driver_id: $driver_id, name: $name, license_number:
$license_number,
                  phone: $phone, rating_avg: 0.0, status: 'available',
password_hash: $password_hash})
CREATE (v:Vehicle {vehicle_id: $vehicle_id, plate_number: $plate_number, model:
$model, color: $color})
CREATE (d)-[:DRIVES]->(v)
CREATE (d)-[:LOCATED_IN]->(z)
```

`POST /api/login` looks the account up by email (rider) or phone (driver) and checks the hash with `check_password_hash`. It returns the same message for an unknown user and for a wrong password, so that it never reveals which accounts exist.

7.2 Feature 2 – Fare Estimate and Ride Request (INSERT)

The API reads the two locations' coordinates, computes the haversine distance, and calculates the fare as **base_fare + per_km_rate × distance**, less the promo percentage. For example, Pondy Bazaar → Kapaleeshwarar Temple is 3.96 km. With Economy that costs 40 + 12 × 3.96 = ₹87.52, or ₹70.02 with WELCOME20. The ride and its four links are created together:

```
MATCH (u:User {user_id: $user_id})
MATCH (t:RideType {ride_type_id: $ride_type_id})
MATCH (pl:Location {location_id: $pickup_id})
MATCH (dl:Location {location_id: $dropoff_id})
CREATE (r:Ride {ride_id: $ride_id, requested_at: $requested_at, status:
'requested',
                distance_km: $distance_km, fare_amount: $fare_amount})
CREATE (u)-[:REQUESTED]->(r)
CREATE (r)-[:OF_TYPE]->(t)
CREATE (r)-[:PICKUP_AT]->(pl)
CREATE (r)-[:DROPOFF_AT]->(dl)
```

7.3 Feature 3 – Nearby Drivers and Open Requests (traversal / JOIN)

Finding the available drivers for a ride takes four hops: Ride → pickup Location → Zone ← Driver → Vehicle. In SQL this would join RIDES, LOCATIONS, ZONES, DRIVERS and VEHICLES.

```
MATCH (:Ride {ride_id: $ride_id})-[:PICKUP_AT]->(:Location)-[:IN_ZONE]->(z:Zone)
      <-[:LOCATED_IN]-(d:Driver {status: 'available'})
```

```
MATCH (d)-[:DRIVES]->(v:Vehicle)
RETURN d.driver_id, d.name, d.rating_avg, v.model, v.plate_number, z.zone_name
ORDER BY d.rating_avg DESC
```

The driver's request list walks the same path in reverse: `(Driver)-[:LOCATED_IN]->(Zone)<-[:IN_ZONE]-(Location)<-[:PICKUP_AT]-(Ride {status:'requested'})`.

7.4 Feature 4 – Accept, Complete, Pay (UPDATE, concurrency-safe)

Acceptance is a single conditional update. The pattern only matches if the ride is still `requested`, the driver is `available`, and both are in the same zone. The zone variable `z` is reused to enforce this. If two drivers press Accept at the same time, the second query finds no match and changes nothing, and the API returns 409 Conflict. This prevents double booking.

```
MATCH (d:Driver {driver_id: $driver_id, status: 'available'})-[:LOCATED_IN]->(z:Zone)
MATCH (r:Ride {ride_id: $ride_id, status: 'requested'})-[:PICKUP_AT]->(:Location)-[:IN_ZONE]->(z)
SET r.status = 'accepted', d.status = 'busy'
CREATE (d)-[:ACCEPTED]->(r)
```

Completing a ride sets it to `completed`, frees the driver and creates a pending Payment in the same transaction. Paying matches only a `pending` payment on the rider's own completed ride, so a ride cannot be paid twice.

```
MATCH (d:Driver {driver_id: $driver_id})-[:ACCEPTED]->(r:Ride {ride_id: $ride_id, status: 'accepted'})
SET r.status = 'completed', d.status = 'available'
CREATE (p:Payment {payment_id: $payment_id, amount: r.fare_amount, method: 'unpaid', status: 'pending'})
CREATE (r)-[:PAID_BY]->(p)
```

7.5 Feature 5 – Rating with Automatic Average

```
MATCH (:User {user_id: $user_id})-[:REQUESTED]->(r:Ride {ride_id: $ride_id, status: 'completed'})
WHERE NOT (r)-[:HAS_RATING]->(:Rating)
CREATE (rt:Rating {rating_id: $rating_id, score: $score, comment: $comment})
CREATE (r)-[:HAS_RATING]->(rt)

MATCH (d:Driver {driver_id: $driver_id})-[:ACCEPTED]->(:Ride)-[:HAS_RATING]->(rt:Rating)
WITH d, avg(rt.score) AS average
SET d.rating_avg = round(average * 100) / 100.0
```

7.6 Feature 6 – Cancellation

A requested or accepted ride can be cancelled by its rider or by its driver. `OPTIONAL MATCH` (the graph form of a LEFT JOIN) finds the driver if there is one, and `FOREACH` makes that driver available again only if they exist:

```
MATCH (r:Ride {ride_id: $ride_id}) WHERE r.status IN ['requested', 'accepted']
OPTIONAL MATCH (d:Driver)-[:ACCEPTED]->(r)
```

```
SET r.status = 'cancelled'
FOREACH (x IN CASE WHEN d IS NULL THEN [] ELSE [d] END | SET x.status =
'available')
CREATE (c:Cancellation {cancellation_id: $cancellation_id, reason: $reason,
cancelled_by: $cancelled_by})
CREATE (r)-[:RESULTED_IN]->(c)
```

7.7 Feature 7 – Admin Statistics (aggregation / GROUP BY)

Cypher groups automatically by the non-aggregated columns. Using `OPTIONAL MATCH` keeps zones with zero rides in the result, just like a LEFT JOIN … GROUP BY.

```
MATCH (r:Ride)
RETURN count(r) AS total_rides,
       sum(CASE WHEN r.status = 'completed' THEN 1 ELSE 0 END) AS completed_rides,
       sum(CASE WHEN r.status = 'completed' THEN r.fare_amount ELSE 0 END) AS
revenue

MATCH (z:Zone)
OPTIONAL MATCH (z)<-[:IN_ZONE]-(:Location)<-[:PICKUP_AT]-(r:Ride)
RETURN z.zone_name AS zone_name, count(r) AS rides ORDER BY rides DESC
```

7.8 Subqueries, Views, Transactions and DELETE

- **Subqueries:** `EXISTS { }`, `COUNT { }` and `CALL { }`, for example rides whose fare is above the overall average (`database/queries.cypher`, section D).
- **View:** Neo4j has no CREATE VIEW. Instead, the reusable `RIDE_DETAIL` query fragment in `queries.py` acts as a view, and every ride query (single ride, rider history, driver history, open requests) appends it.
- **Transactions:** every API call runs in a single transaction through `execute_query`. Multi-step operations such as complete-and-create-payment therefore either commit completely or not at all. `database/queries.cypher` also shows explicit `:begin` / `:commit` / `:rollback` and `CALL { } IN TRANSACTIONS`.
- **DELETE:** `DETACH DELETE`, which works like ON DELETE CASCADE, is used by the test clean-up and by `seed.py --reset`.

Operation	Where used
INSERT (CREATE / MERGE)	register, request ride, complete (payment), rate, cancel, seed data
SELECT (MATCH … RETURN)	login, lookups, ride detail, history, nearby drivers, open requests, stats
UPDATE (SET)	accept, complete, pay, cancel (driver freed), rating average
DELETE (DETACH DELETE)	test clean-up, seed --reset, queries.cypher A9
JOIN (multi-hop MATCH)	nearby drivers, open requests, RIDE_DETAIL
GROUP BY / aggregates	admin stats, earnings, rating average
LEFT JOIN (OPTIONAL MATCH)	RIDE_DETAIL, rides per zone, cancel

Table 7.1: Database operations and where they are used

8. Testing, Results and Discussion

8.1 Testing Approach

- **Automated end-to-end tests:** `tests/test_api.py` calls the running API through a complete ride life cycle (register → login → estimate → request → accept → complete → pay → rate → cancel → history → admin) and checks every response. It prints PASS or FAIL for each check and a total, and it deletes the data it created at the end.
- **Negative tests:** duplicate values, missing fields, invalid values, missing records, unauthorised actions, repeated actions and a database outage.
- **Manual UI testing:** the full flow in two browser windows (rider and driver), with the results checked in Neo4j Browser.

8.2 Test Cases

The table below lists the main test cases. All forty are in `tests/TEST_CASES.md`. The automated script ran 73 checks on 01-11-2026 and all 73 passed (Figure 8.9). TC39 was observed when the Neo4j instance was stopped: every API call returned 503 with the message "Cannot connect to the Neo4j database". TC40 was run against the database, which rejected the duplicate `user_id` with `Neo.ClientError.Schema.ConstraintValidationFailed`. All three database scripts (`schema.cypher`, `sample_data.cypher`, `queries.cypher`; 156 statements) also ran without errors, and the SQL scripts ran with only the five intended constraint violations rejected.

ID	Feature	Input	Expected	Actual
TC01	Lookup data	GET locations, zones, ride types	15, 6 and 4 records	Pass
TC04	Register rider	Valid new rider	201 Created	Pass
TC05	Duplicate email	Same email again	409 Conflict	Pass
TC06	Missing fields	Only a name	400 "'email' is required"	Pass
TC09	Wrong password	Valid email, wrong password	401, generic message	Pass
TC10	Unknown user	Unknown email	401, same message	Pass
TC13	Fare estimate	L1 → L4, Economy	base + per_km × distance	Pass
TC14	Promo code	Same + WELCOME20	Exactly 20% less	Pass
TC15	Same pickup and dropoff	pickup = dropoff	400	Pass
TC16	Missing record	Unknown user_id	404 Not found	Pass
TC17	Request ride	Valid request	201, status requested	Pass
TC19	Zone matching	Driver in other zone	Ride not visible	Pass
TC20	Wrong-zone accept	Other-zone driver accepts	403	Pass
TC21	Accept ride	Same-zone driver	200, driver busy	Pass
TC22	Double accept	Accept again	409	Pass
TC24	Complete ride	Accepting driver	Pending payment created	Pass
TC25	Invalid input	Payment method "bitcoin"	400	Pass

ID	Feature	Input	Expected	Actual
TC27	Double payment	Pay again	409	Pass
TC29	Double rating	Rate again	409	Pass
TC30	Invalid score	score = 9	400	Pass
TC33	Cancel ride	Rider cancels accepted ride	Cancelled, driver freed	Pass
TC38	Admin stats	GET /api/admin/stats	Totals, revenue, 6 zones	Pass
TC39	Database down	Neo4j stopped	503 with message	Pass
TC40	Constraint violation	Duplicate user_id in Neo4j	ConstraintValidationFailed	Pass

Table 8.1: Test cases with expected and actual outcomes

8.3 Results on the Sample Data

On a freshly seeded database, the admin dashboard and the queries in Section 7 return the following values. They can be checked with `database/queries.cypher`, section C. The screenshots in Section 8.4 were taken on the team's working database, which also contains a few rides created while testing, so the dashboard there shows slightly higher totals.

Measure	Value
Total rides / completed / cancelled	24 / 21 / 3
Revenue from completed rides	₹1,973.40
Rides per pickup zone	T. Nagar 9, Mylapore 4, Anna Nagar 3, Velachery 3, Chennai Central 3, Guindy 2
Revenue by ride type	Premium ₹954.86 (6), Economy ₹757.20 (9), Auto ₹202.50 (4), Bike ₹58.84 (2)
Payment methods	UPI 10, Cash 6, Card 5
Demo driver Praveen (D11)	4 completed rides, ₹553.56 earned, rating 4.75

Table 8.2: Results on the seeded data

8.4 Screenshots

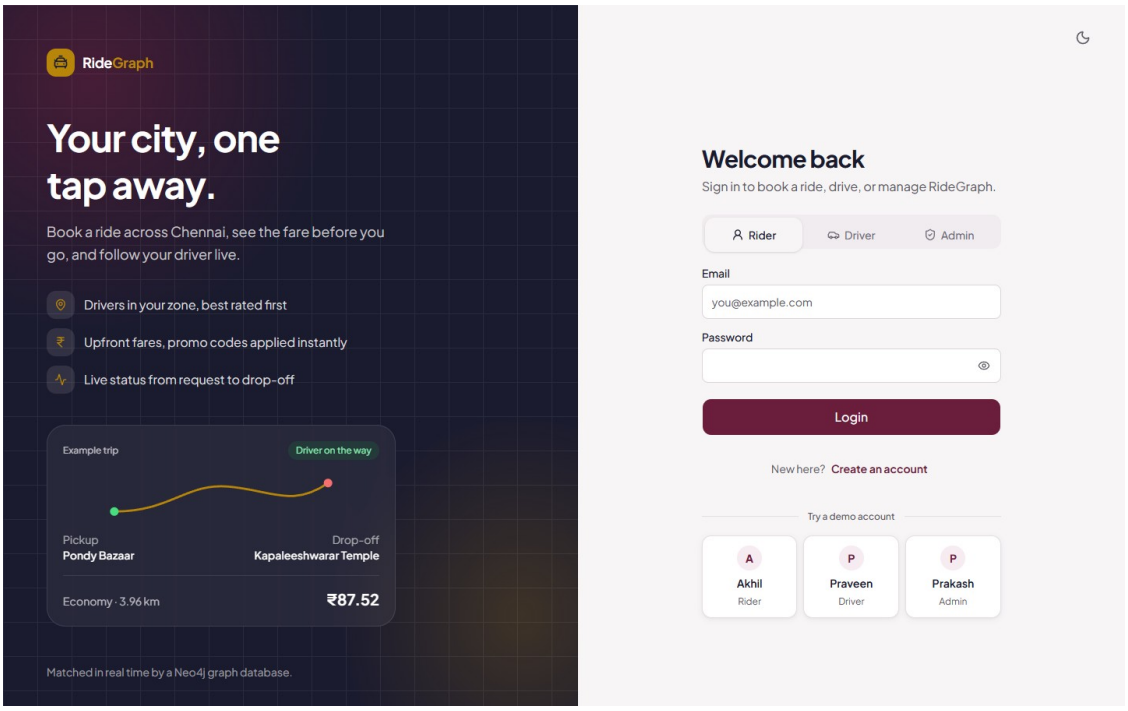


Figure 8.1: Login page with Rider / Driver / Admin tabs and demo accounts

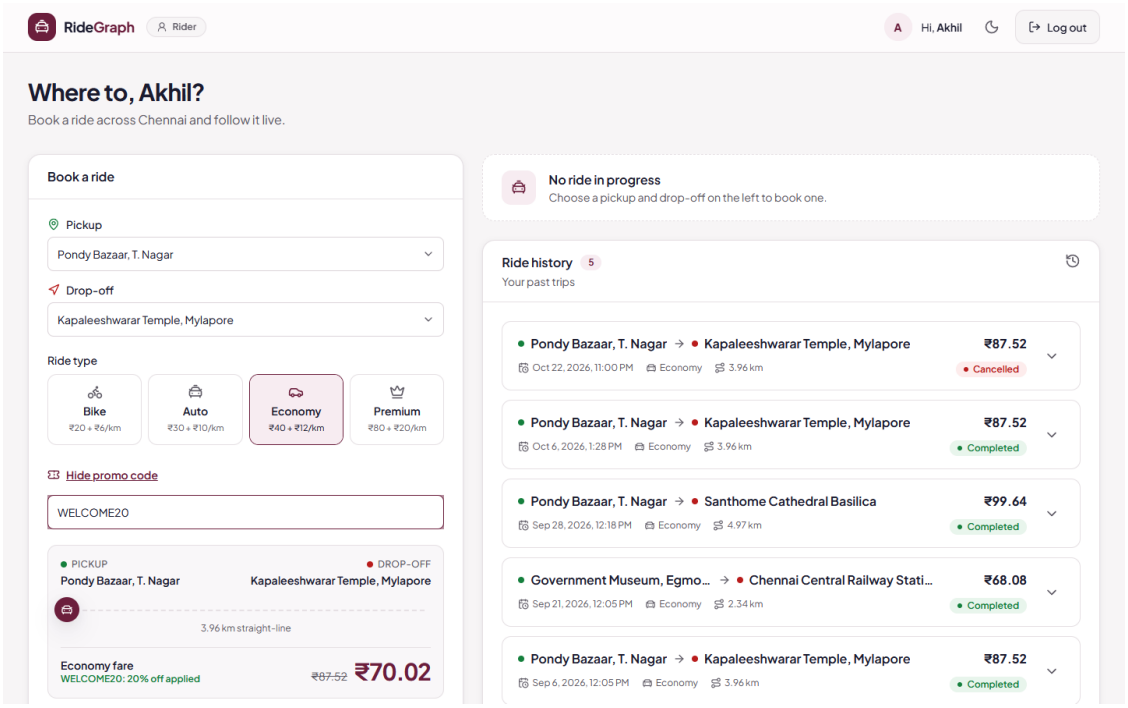


Figure 8.2: Fare estimate – Pondy Bazaar → Kapaleeshwarar Temple, Economy, WELCOME20: ₹87.52 → ₹70.02

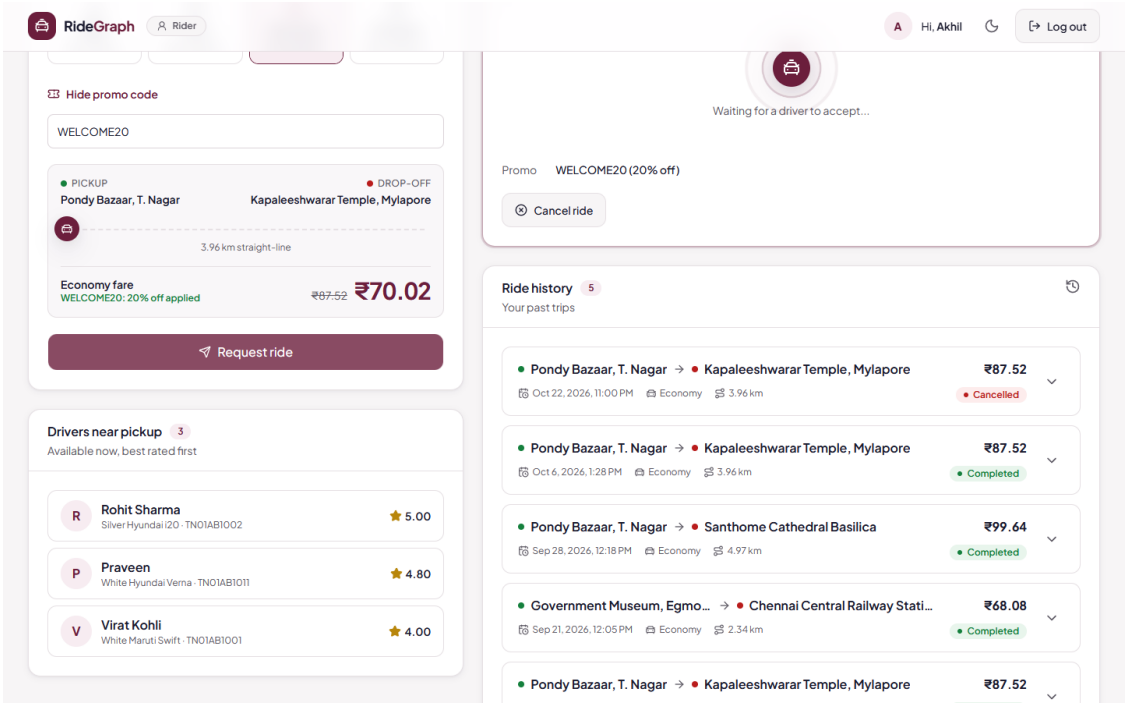


Figure 8.3: Ride requested – available drivers in the pickup zone, best rated first

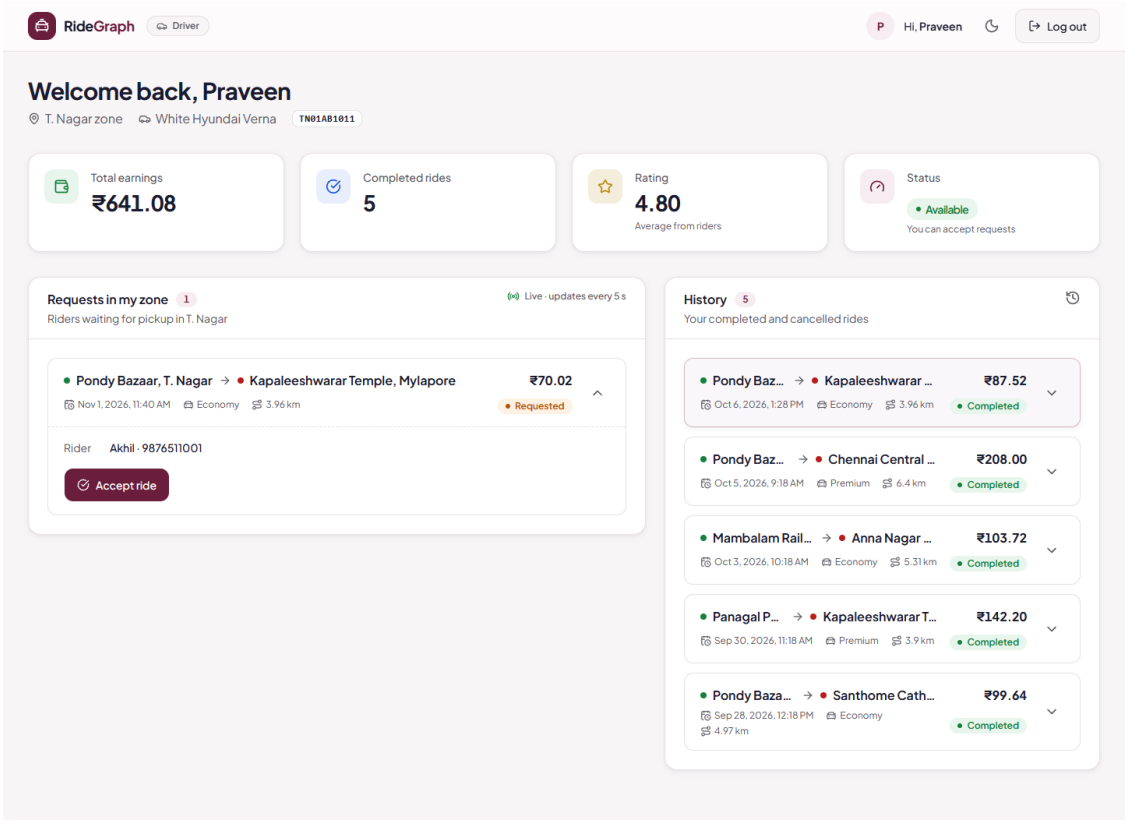


Figure 8.4: Driver Praveen sees the open request in his zone (T. Nagar)

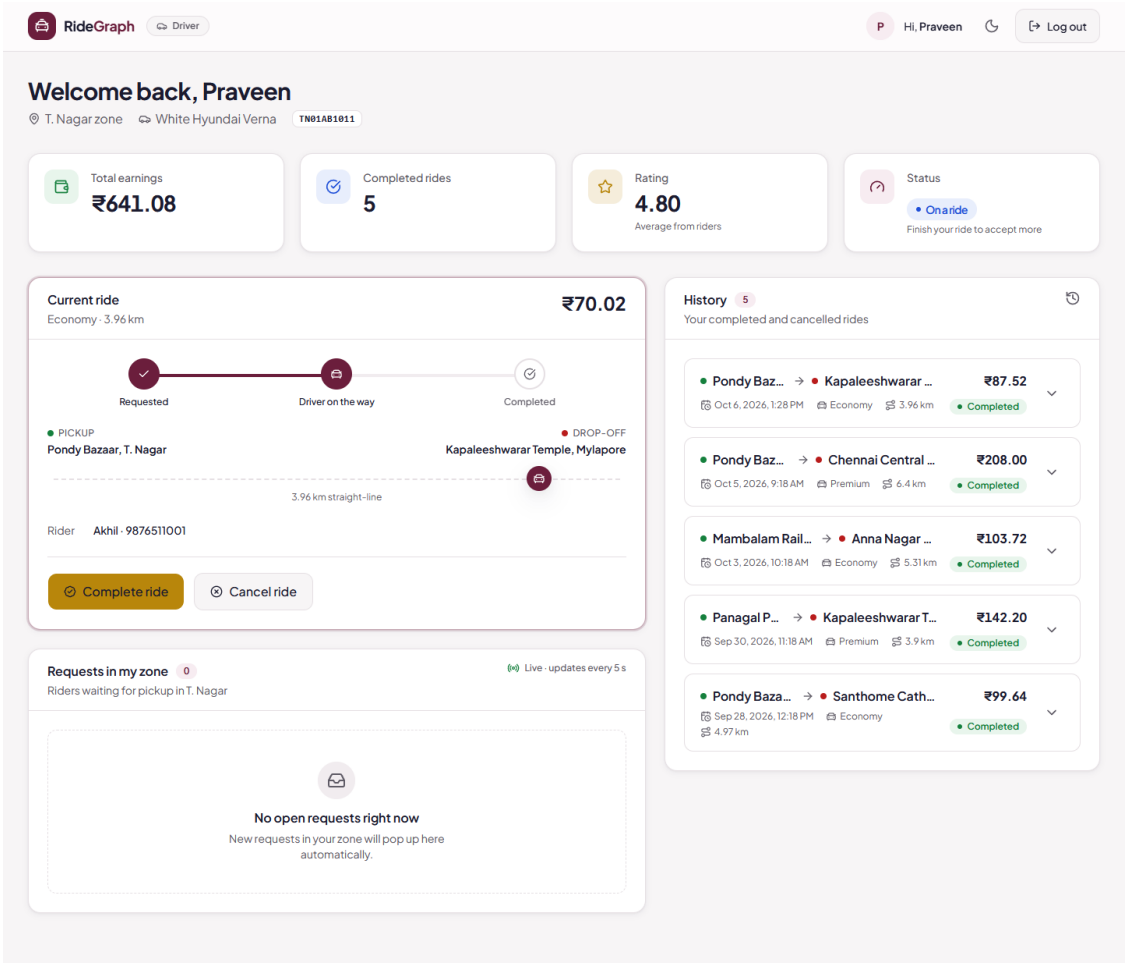


Figure 8.5: After Accept – driver status On a ride, ride stepper at Driver on the way

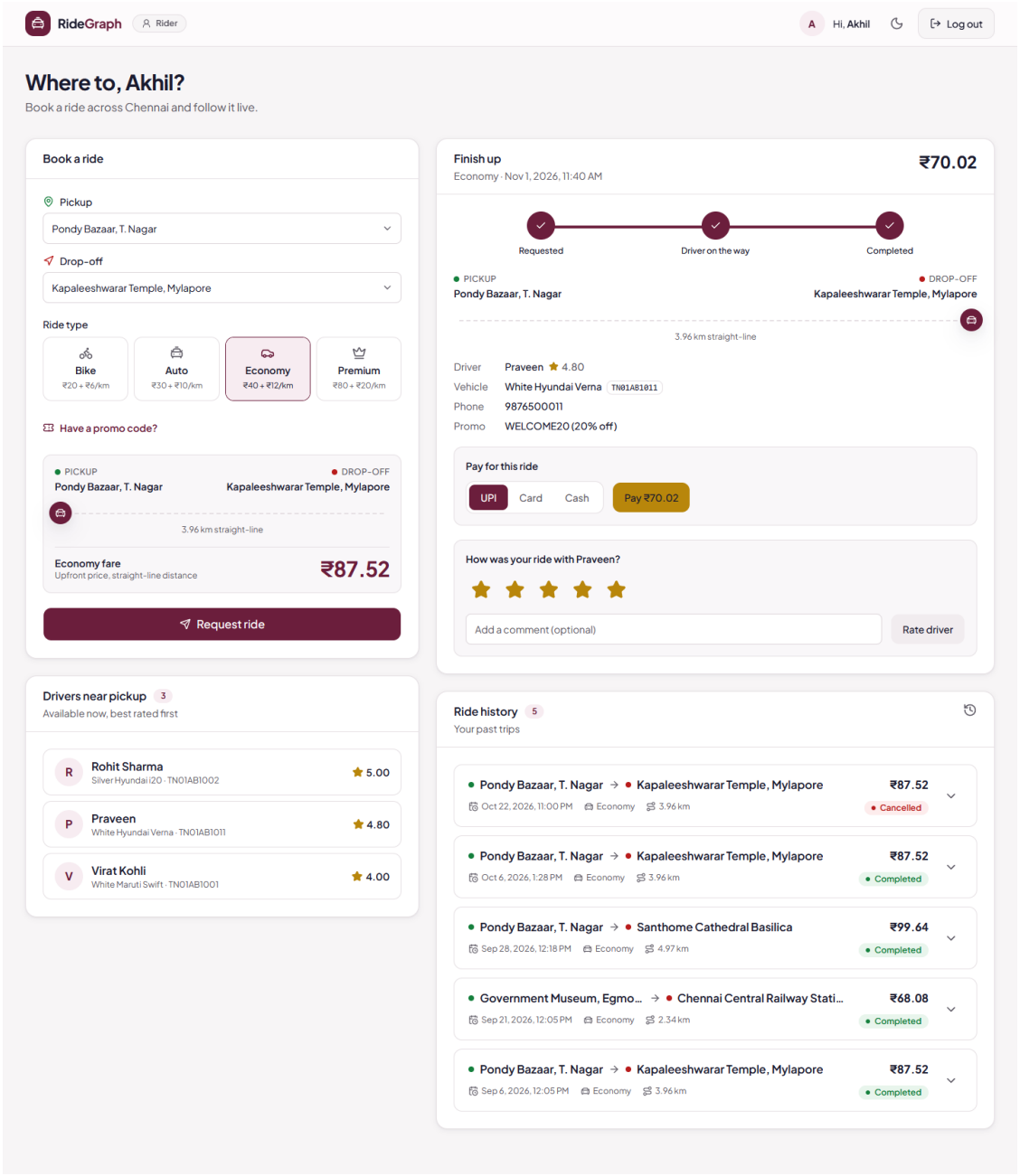


Figure 8.6: Ride completed – rider pays (UPI / Card / Cash) and rates the driver

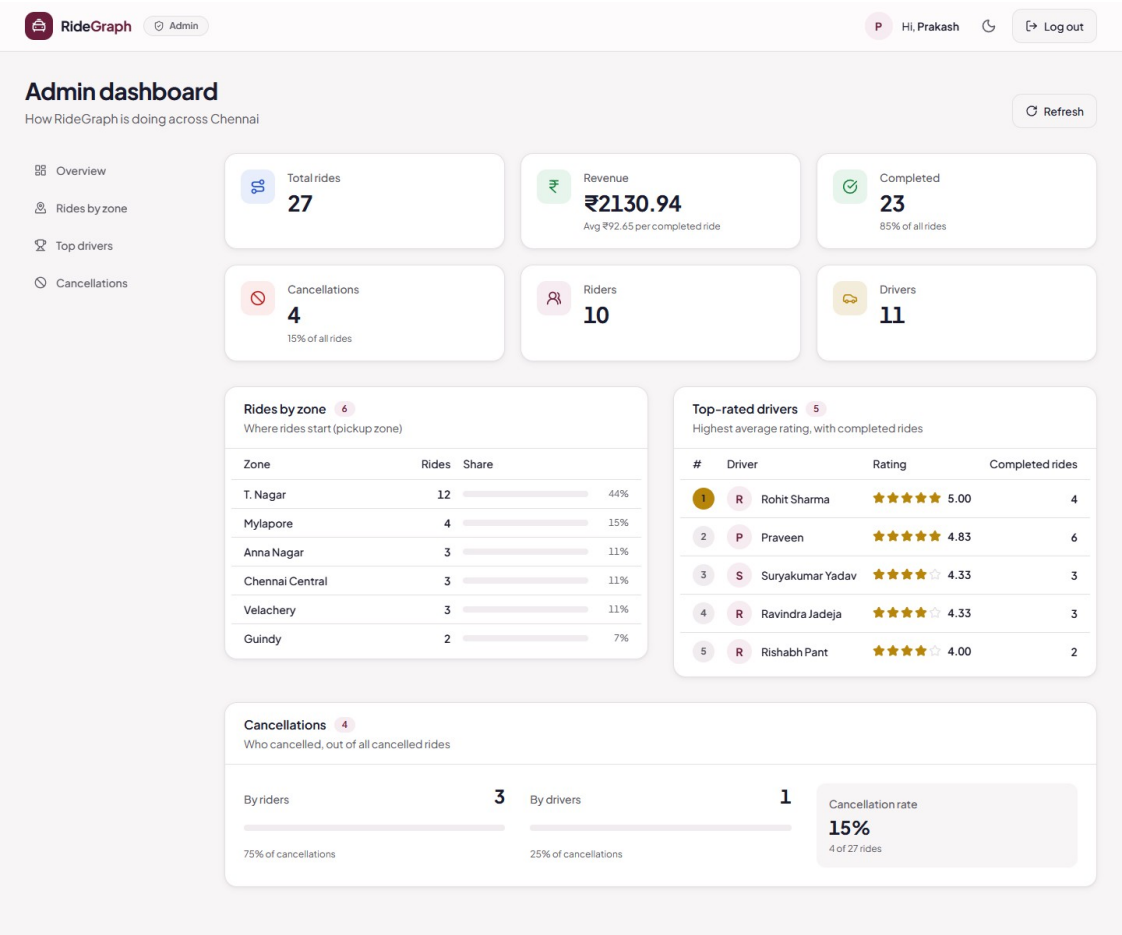


Figure 8.7: Admin dashboard – totals, revenue, rides by zone, top drivers, cancellations

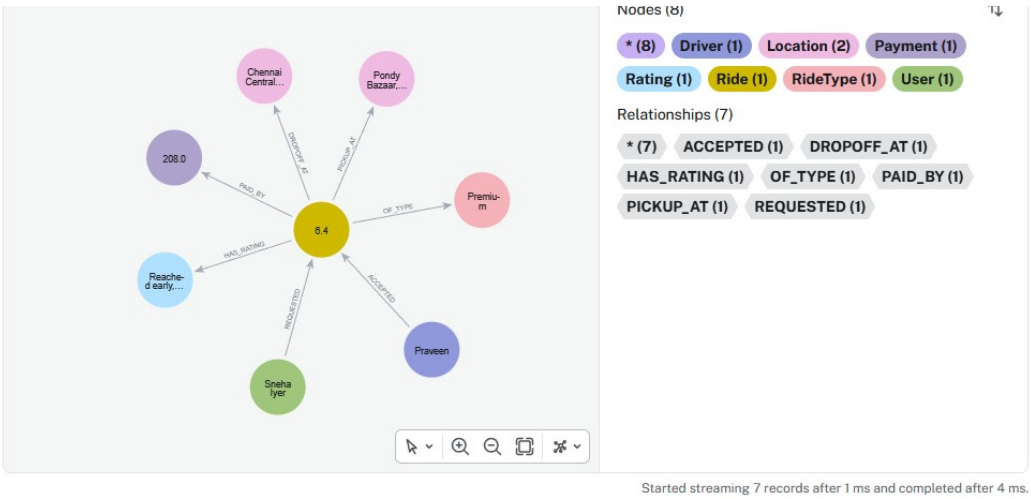


Figure 8.8: Neo4j Browser – ride R24 with its rider, driver, type, locations, payment and rating

```
Command Prompt — venv\Scripts\python tests\test_api.py

> python tests\test_api.py
== Static frontend ==
  PASS  GET / serves index.html
== Drivers: Praveen (demo driver) and Virat Kohli ==
  PASS  exactly 11 drivers in the database
  PASS  Praveen (D11) can log in
  PASS  Praveen: available, T. Nagar zone, own vehicle
  PASS  Praveen rating is high (seed value is about 4.7)
  PASS  Praveen has at least his 4 history rides and earnings
  PASS  Praveen's 4 history rides (R21-R24) all have payment + rating
  PASS  Praveen shows up in nearby drivers
  PASS  nearby drivers are sorted by rating
  PASS  Virat Kohli (D1) still exists and can log in
== Lookup data ==
  PASS  GET /api/locations (15)
  PASS  GET /api/zones (6)
  PASS  GET /api/ride-types (4)
  PASS  GET /api/promo/<code>
  PASS  invalid promo -> 404
== Register / login ==
  PASS  register rider -> 201
  PASS  duplicate email -> 409
  PASS  missing fields -> 400
  PASS  register driver -> 201
  PASS  login new rider
  PASS  wrong password -> 401
  PASS  login demo rider
  PASS  login demo driver (T. Nagar)
  PASS  login admin
  PASS  bad admin password -> 401
== Login errors and renamed drivers ==
  PASS  wrong password -> 401 + message
  PASS  wrong username -> 401, SAME message
  PASS  unknown driver phone -> 401, same message
  PASS  driver wrong password -> 401, same message
  PASS  admin wrong password -> 401, same message
  PASS  empty password -> 400
  PASS  missing username -> 400
  PASS  missing both fields -> 400
  PASS  renamed driver D1 logs in as Virat Kohli
  PASS  all 10 drivers log in with their cricketer names
  PASS  duplicate email message
== Fare estimate and ride request ==
  PASS  fare estimate
  PASS  fare = base + per_km * distance
  PASS  promo gives 20% off
  PASS  same pickup/dropoff -> 400
  PASS  bad promo on request -> 404
  PASS  unknown rider -> 404
  PASS  nearby drivers by location
  PASS  request ride -> 201
  PASS  ride fare matches estimate
  PASS  nearby drivers sorted by rating
  PASS  driver sees request in own zone
  PASS  driver in other zone does NOT see it
== Accept / complete / pay / rate ==
  PASS  driver from other zone cannot accept -> 403
  PASS  accept ride
  PASS  driver is now busy
  PASS  accepting twice -> 409
  PASS  wrong driver cannot complete -> 403
  PASS  complete ride creates pending payment
  PASS  driver free again
  PASS  bad payment method -> 400
  PASS  pay ride
  PASS  paying twice -> 409
  PASS  rate ride -> 201
  PASS  rating twice -> 409
  PASS  score 9 -> 400
  PASS  driver rating_avg updated
== Cancellation flow ==
  PASS  accept second ride
  PASS  driver busy again
  PASS  someone else's cancel -> 403
  PASS  cancel ride
  PASS  driver freed after cancel
  PASS  cancel twice -> 409
  PASS  cancel un-accepted ride
== History, earnings, admin ==
  PASS  rider history
  PASS  driver rides + earnings
  PASS  unknown user history -> 404
  PASS  admin stats
(test data cleaned up)

73 passed, 0 failed
```

Figure 8.9: Output of tests/test_api.py – 73 passed, 0 failed

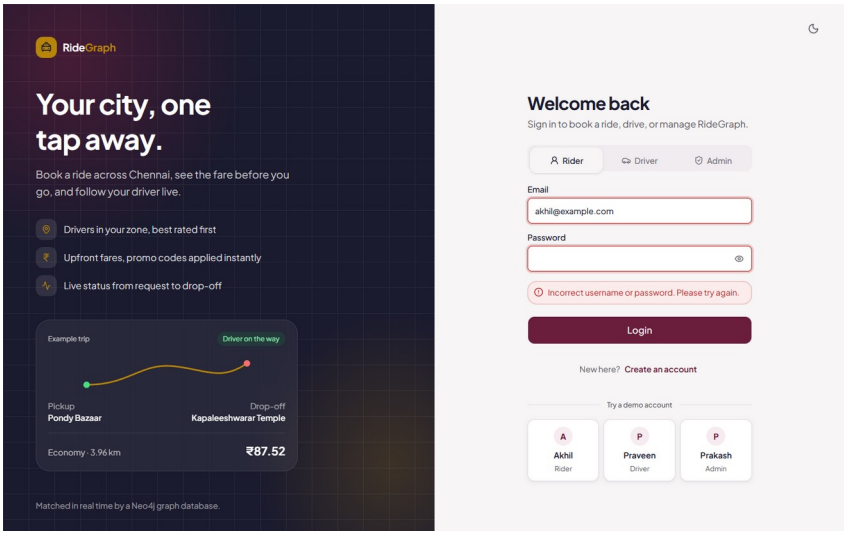


Figure 8.10: Wrong password – one generic message

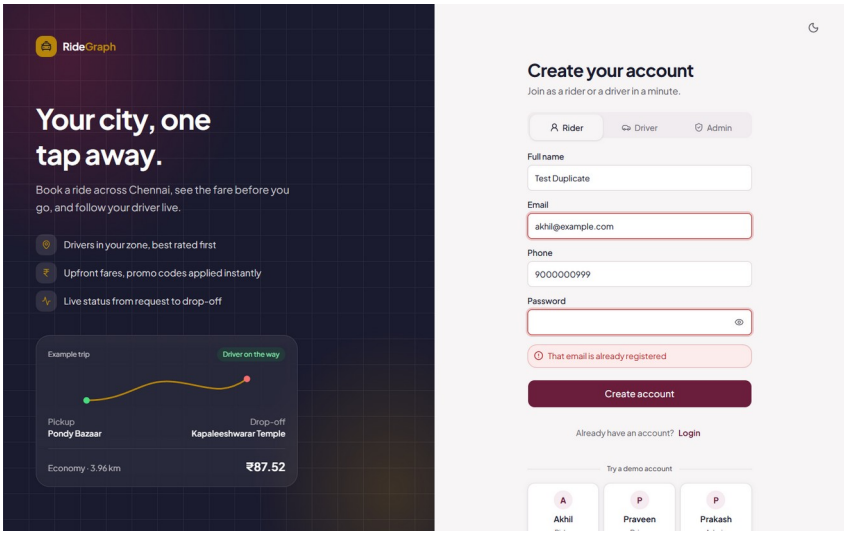


Figure 8.11: Registering an existing email – 409 "That email is already registered"

8.5 Discussion

The graph model made the relationship-heavy features short and readable. Nearby drivers, open requests and ride details are each one `MATCH` pattern instead of a chain of joins. The conditional-update pattern turned out to be the simplest way to enforce business rules such as no double acceptance, pay once and rate once: if the pre-condition no longer holds, the pattern matches nothing and the API reports a conflict, without separate locking code.

There were also trade-offs. Neo4j Community does not support property-existence or type constraints, so domain rules had to move into the API layer. Aggregate reports such as revenue or rides per zone are equally easy in SQL. The graph approach gives the biggest advantage for multi-hop questions such as repeat drivers for a rider, and for extending the model with new relationship types without changing the schema.

9. Conclusion, Limitations and Future Enhancements

9.1 Conclusion

RideGraph shows a complete database project carried from requirements to a working application. The ER design was converted to a relational schema, normalized to 3NF/BCNF with deliberate and justified denormalization, and implemented as a Neo4j property graph with uniqueness constraints and indexes. A three-tier React–Flask–Neo4j application provides seven end-to-end features covering every CRUD operation, multi-hop traversals, aggregation and subqueries. The application validates input, returns clear errors for duplicates, missing records and rule violations, and was checked with documented and automated tests.

9.2 Limitations

- Authentication is session-less. The API trusts the user and driver ids sent by the browser, and the admin endpoint has no token check.
- Distances are straight-line (haversine), not road distances, and there is no live GPS or map. A driver's position is their zone.
- Payments are simulated, with no payment gateway.
- Property-existence and type constraints need Neo4j Enterprise, so domain rules are enforced in the API.
- Ratings go one way only, from rider to driver.

9.3 Future Enhancements

- JWT-based authentication with role checks on every endpoint.
- Live driver locations using Neo4j `point()` and distance indexes, shown on a map, with road distances from a routing API.
- Shortest-path and route queries with Neo4j Graph Data Science, and surge pricing based on demand per zone.
- Two-way ratings, ride scheduling, carpooling (a new SHARES relationship) and notifications.
- Integration with a real payment gateway (UPI/card) and deployment to Neo4j AuraDB.

References

[1] R. Elmasri and S. B. Navathe, Fundamentals of Database Systems, 7th ed. Boston, MA, USA: Pearson, 2016.

[2] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill, 2020.

[3] I. Robinson, J. Webber, and E. Eifrem, Graph Databases: New Opportunities for Connected Data, 2nd ed. Sebastopol, CA, USA: O'Reilly Media, 2015.

[4] R. Angles and C. Gutierrez, "Survey of graph database models," ACM Computing Surveys, vol. 40, no. 1, pp. 1–39, Feb. 2008.

[5] N. Francis et al., "Cypher: An evolving query language for property graphs," in Proc. ACM SIGMOD Int. Conf. Management of Data, Houston, TX, USA, 2018, pp. 1433–1445.

[6] Neo4j, Inc., "Neo4j Cypher Manual." [Online]. Available: https://neo4j.com/docs/cypher-manual/current/

[7] Neo4j, Inc., "Neo4j Python Driver Manual." [Online]. Available: https://neo4j.com/docs/python-manual/current/

[8] Neo4j, Inc., "Constraints – Cypher Manual." [Online]. Available: https://neo4j.com/docs/cypher-manual/current/constraints/

[9] Pallets Projects, "Flask Documentation." [Online]. Available: https://flask.palletsprojects.com/

[10] Pallets Projects, "Werkzeug – Utilities: Password hashing." [Online]. Available: https://werkzeug.palletsprojects.com/

[11] Meta Platforms, Inc., "React Documentation." [Online]. Available: https://react.dev/

[12] Vite, "Vite Documentation." [Online]. Available: https://vite.dev/

[13] R. W. Sinnott, "Virtues of the Haversine," Sky and Telescope, vol. 68, no. 2, p. 159, 1984.

[14] Uber Technologies, Inc. [Online]. Available: https://www.uber.com/in/en/

[15] ANI Technologies Pvt. Ltd. (Ola). [Online]. Available: https://www.olacabs.com/

[16] Roppen Transportation Services Pvt. Ltd. (Rapido). [Online]. Available: https://www.rapido.bike/